Hate Speech Detection

Jami Sidhava
202201038
202201038@dau.ac.in

I. Abstract

***Abstract*—The rise of toxic language on social media platforms necessitates robust hate speech detection systems. This paper presents a hybrid model pipeline combining classical machine learning techniques, deep learning with word embeddings, and transformer-based models to identify hateful content. We relabeled a publicly available dataset using a transformer model to correct inconsistencies. Our final ensemble combines predictions from GRU, RoBERTa, and ToxiGen-RoBERTa to maximize accuracy and robustness, especially against subtle hate speech, sarcasm, and slang.The final ensemble model achieved strong performance across all evaluation metrics, surpassing individual models in accuracy and robustness. It effectively captured both explicit and subtle hate speech, including sarcasm and implicit bias. To enable public access, the model was deployed as a web application on Hugging Face Spaces for real-time interaction and testing.**

Index Terms—Hate Speech Detection, Natural Language Processing, Deep Learning, Transformers, GRU, RoBERTa, Ensemble Learning.

II. Introduction

The rise of social media has transformed global communication but also intensified the spread of harmful content, particularly hate speech. Defined as abusive or threatening language targeting individuals or groups based on identity traits like race, religion, gender, or nationality, hate speech poses serious risks to mental health, social harmony, and public safety.

Manual moderation of such content is time-consuming, inconsistent, and impractical at scale. Platforms process billions of posts daily, far beyond the capacity of human moderators. Furthermore, the subjective and culturally sensitive nature of language makes moderation prone to bias and error.

Automated hate speech detection has become essential, yet it remains challenging due to the subtle, implicit, and context-dependent nature of hateful content. Sarcasm, coded language, and slang often obscure intent, making detection difficult for conventional algorithms.

This paper presents a robust ensemble-based approach combining classical machine learning, GloVe-based deep learning, and transformer models like RoBERTa and ToxiGen. By leveraging the strengths of each paradigm, our system captures both explicit and nuanced hate speech. The final model is trained on a re-labeled dataset for consistency and deployed on Hugging Face for public use, enabling scalable, real-time moderation.

III. Related Work

Mollas et al. observed that treating hate speech as a simple binary task overlooks its nuanced manifestations, such as calls for violence, targeted versus generalized insults, and various protected categories. To address this, they introduced the ETHOS dataset, which pools comments from YouTube and Reddit, then applies active-learning, driven sampling and crowdsourced multi-label annotation (incitement, target type, protected trait). Their balanced, fine-grained corpus, released both as a binary and an eight-label multi-label CSV, enables richer model training and benchmarking, with transformer models (BERT, DistilBERT) outperforming classical classifiers and specialized BiLSTM+attention nets excelling in the multi-label setting.

Recognizing ethical shortcomings in reactive content removal, another line of work proposes "quarantining" flagged posts rather than outright deletion. In this model, an automated detector holds suspect messages in isolation and alerts only the sender and recipient via a severity gauge ("Hate-O-Meter"). Depending on their consent, the content is released or remains hidden; extensions include multipartite alerts for indirect viewers and user-configurable filters. This approach seeks a middle ground between censorship and free expression, granting end users controlled exposure while preserving speakers' rights.

Zimmerman et al. demonstrated that ensembles of CNNs, each initialized differently and averaged at the softmax level, yield more stable and accurate hate-speech classifiers across both hate-speech and sentiment datasets. Their experiments showed consistent F1 improvements (2%) and reduced variance compared to standalone models. Separately, Cortiz and Zubiaga warn that even high-performing transformers (e.g., BERT fine-tuned for Brazilian Portuguese) can perpetuate bias or lack transparency. They advocate limiting AI to flagging content, with final decisions left to human reviewers, to balance efficiency with ethical safeguards, especially around free speech and privacy.

IV. Dataset and Preprocessing

A. Dataset

Our research utilized the "Hate Speech Detection Curated Dataset" obtained from Kaggle, which represents one of the most comprehensive publicly available datasets for hate speech research. Dataset initially included English and Serbo-Croatian tweets.Original Dataset consisted of 10 Lakh tweets, after manually removing the non-english tweets we were left with some 7 Lakh odd tweets. Label noise was observed, where some hate speech samples were incorrectly labeled as non-hate. To resolve this, we used the toxic BERT transformer model to relabel the dataset, significantly improving data quality.

B. Text Preprocessing

To prepare the dataset for modeling, we applied a standardized text preprocessing pipeline using widely adopted Natural Language Processing (NLP) techniques. The primary goal was to clean and normalize the raw text to reduce noise and ensure consistency across all models.

The preprocessing steps included:

- **Lowercasing:** Converted all text to lowercase to unify word variants and decrease the total vocabulary size.
- **URL/HTML (Noise Removal):** Eliminated irrelevant elements such as punctuation, URLs, emojis, and HTML tags to clean the raw input.
- **Stopword Removal:** Removed frequent but uninformative words (e.g., "the", "is", "at") using NLTK's standard stopword list.
- **Lemmatization:** Transformed words into their basic dictionary forms to normalize variations and improve consistency.
- **Tokenization:** Broke cleaned text into individual tokens or words, preparing it for feature extraction or embedding.

These steps were automated via a custom `clean_text()` function, ensuring consistent preprocessing across all models for training and evaluation.

V. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to better understand the structure and characteristics of the dataset. After re-labeling using a transformer model, we observed a near-balanced distribution of hate and non-hate tweets, which is essential for fair model evaluation.

We analyzed the word count across classes, revealing that hateful tweets are typically shorter but tend to contain stronger, more emotionally charged language. These findings suggest that hate speech is often direct and concise.

To visualize the distinct language used in both categories, we generated WordClouds for hateful and non-hateful tweets. The hateful tweets prominently featured abusive and targeted terms, while non-hateful tweets contained more general, conversational vocabulary.

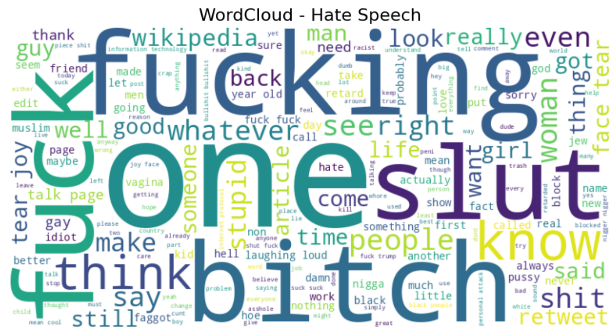

Fig. 1: WordCloud for Hateful Tweets

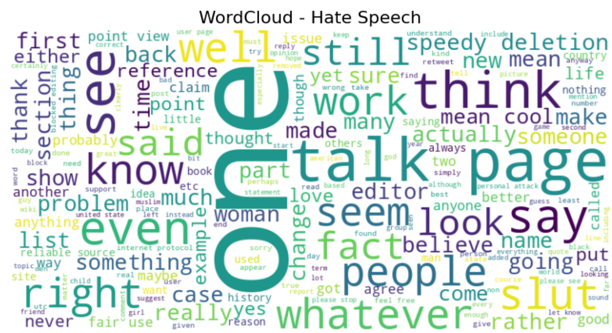

Fig. 2: WordCloud for Non-Hateful Tweets

These visualizations highlight the linguistic differences between the two classes and support feature engineering and model interpretation.

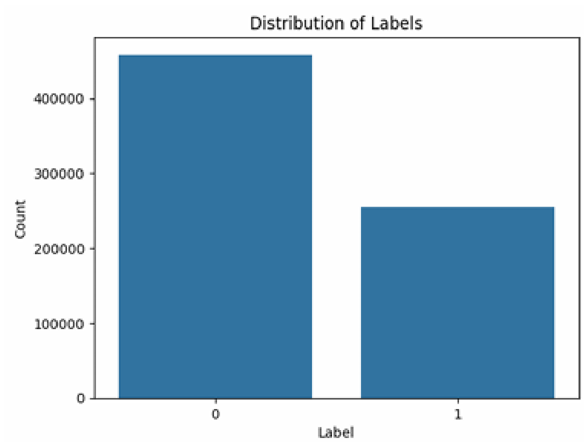

Fig. 3: Distribution of Hate and Non-Hate Tweets

Next, we examined tweet length distributions by counting the number of words in each tweet. Hateful tweets were generally shorter, suggesting more direct language, whereas non-hateful tweets exhibited a wider range in length and often included more neutral conversational content.

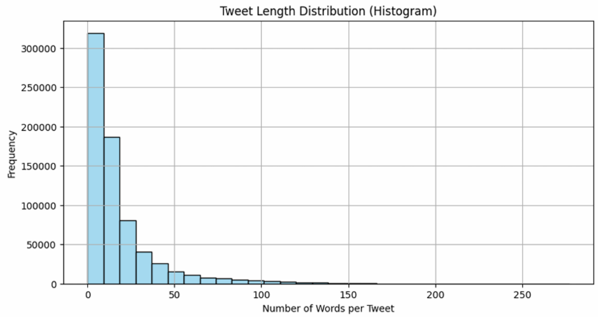

Fig. 4: Tweet Length Distribution Across Classes

VI. Methodology

This section outlines the key components of our modeling pipeline, including feature extraction, classical machine learning approaches, and a deep learning model incorporating GloVe embeddings and GRU layers.

A. Feature Engineering

To prepare the text for classical machine learning models, we used TF-IDF vectorization. This method converts each piece of text into a set of numbers that represent how important each word is in that text compared to the rest of the dataset. To keep things efficient, we only kept the 10,000 most common words. These TF-IDF features were then used as input for our machine learning classifiers.

B. Classical Machine Learning Models

We experimented with several classical algorithms, each suited for different learning assumptions:

- **Logistic Regression:** A linear model ideal for binary classification. It performed well on linearly separable TF-IDF features and demonstrated high precision and recall.
- **Naive Bayes:** Based on the assumption of word independence, this model was computationally efficient and fast to train but slightly less accurate due to its simplicity.
- **Support Vector Machine (LinearSVC):** This model excelled in handling high-dimensional feature spaces and outperformed others in terms of F1-score and decision margin.

All classical models were trained using stratified 5-fold cross-validation to ensure robust performance and minimize variance due to data splits.

TABLE I: Performance of Classical Machine Learning Models

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	94.08%	94.71%	88.39%	91.44%
Naive Bayes	86.83%	82.98%	79.50%	81.20%
LinearSVC	**94.52%**	**94.39%**	**90.04%**	**92.16%**

C. Deep Learning: GloVe + GRU

To capture deeper semantic relationships within the text, we implemented a deep learning model based on Bidirectional Gated Recurrent Units (GRU) using pre-trained GloVe embeddings. We utilized GloVe vectors with 100 dimensions, which encode rich semantic information and help generalize across similar words in different contexts.

The architecture of the network is as follows:

D. Deep Learning Architecture: GloVe + GRU

Our deep learning model is designed to capture both the meaning of individual words and their context within a sentence. The architecture includes the following layers:

- **Embedding Layer:** Uses pre-trained GloVe vectors (100 dimensions) to convert words into meaningful numerical representations.
- **Bidirectional GRU Layer:** A GRU with 128 units that reads the text from both directions to better understand the full meaning of each sentence.
- **Global Max Pooling:** Picks out the most important features from the GRU output, helping reduce the data size while keeping key information.
- **Dense Layer:** A fully connected layer with ReLU activation that helps the model learn more complex patterns.
- **Dropout Layer:** Randomly turns off some neurons during training to prevent the model from memorizing and overfitting.
- **Output Layer:** A single neuron with a sigmoid activation that gives the final prediction: whether the text is hateful or not.

The model was trained using the Adam optimizer and binary cross-entropy loss. We used early stopping to prevent overfitting by stopping training when the model stopped improving on the validation set. This setup helped the model learn subtle and hidden forms of hate speech effectively.

E. Transformer Models

We fine-tuned two transformer-based models for hate speech detection:

- **RoBERTa-base:** A refined version of BERT that was trained using more data and a better training method, helping it understand context and meaning in text more effectively.
- **ToxiGen-RoBERTa:** Fine-tuned specifically for detecting toxic and adversarial language, making it effective for subtle hate speech.

Both models use the [CLS] token's contextual embedding, which is passed through a dropout layer and a fully connected dense layer for binary classification. These models capture nuanced language patterns and significantly enhance detection of implicit or sarcastic hate speech.

F. Ensemble Method

To leverage the complementary strengths of different models, we implemented a weighted ensemble combining GRU, RoBERTa, and ToxiGen-RoBERTa outputs. The final prediction score is computed as:

$$P_{score} = 0.2 \cdot \text{GRU} + 0.45 \cdot \text{RoBERTa} + 0.35 \cdot \text{ToxiGen} \quad (1)$$

Predictions with $P_{score} > 0.5$ are classified as **HATE** speech. This ensemble strategy significantly improved performance, particularly in detecting subtle, implicit, or sarcastic hate speech that individual models often miss.

VII. Evaluation and Results

All models were evaluated based on Accuracy. The ensemble model achieved the best performance, benefiting from the combined strengths of classical ML, deep learning, and transformer models.

TABLE II: Model Comparison and Accuracy

Model	**Text Representation**	**Model Type**	**Accuracy**
Logistic Regression	TF-IDF	Classical ML	94.08%
Naive Bayes	TF-IDF	Classical ML	86.83%
SVM (LinearSVC)	TF-IDF	Classical ML	94.52%
GloVe + GRU	GloVe Embedding	Deep Neural Network	95.80%
RoBERTa / ToxiGen	Transformer Encoding	Transformer	∼96.00%
Ensemble (Final)	Mixed	Hybrid	**96.5%+**

TABLE III: Computational Performance Comparison

Model	**Training Time**	**Inference Time**	**Model Size**
Logistic Regression	2.3 min	0.8 ms	12 MB
Naive Bayes	1.1 min	0.5 ms	8 MB
SVM (LinearSVC)	4.7 min	1.2 ms	15 MB
GRU Model	45 min	15.2 ms	67 MB
RoBERTa	180 min	45.8 ms	355 MB
ToxiGen-RoBERTa	165 min	42.3 ms	340 MB
Ensemble Model	**390 min**	**103.3 ms**	**712 MB**

VIII. Deployment

To make our system accessible and interactive, we deployed the final ensemble model on **Hugging Face Spaces**, a platform that allows seamless model hosting with a user-friendly web interface. Users can input any text and instantly receive a classification — identifying whether the content is *Hateful* or *Not Hateful* — without needing to install any software.

The deployment stack consists of a lightweight **FastAPI** backend that handles requests and loads the pre-trained ensemble model along with its tokenizer and weights. The frontend is a simple HTML interface that communicates with the FastAPI server, enabling real-time inference through HTTP calls. This setup ensures both speed and scalability while making the model publicly accessible for real-world experimentation and feedback.

Live Demo Link: https://aaryan24-hate-speech-detector.hf.space/?text=

IX. Conclusion

This study demonstrates that combining classical ML, deep learning, and transformers leads to a robust hate speech detection system. The ensemble approach balances simplicity, accuracy, and contextual understanding. Our deployment shows real-world applicability, making it a valuable tool for social platforms and content moderators

Acknowledgment

We express our sincere gratitude to the following contributors and supporters:

- **Prof. Purbasha Das** — for her continuous mentorship and guidance throughout the research internship.
- **Open-source community** — for providing access to essential datasets, libraries, and pre-trained models that enabled our research and experimentation.
- **Kaggle community** — specifically for the "Hate Speech Detection Curated Dataset," which served as the foundation for our model development and evaluation.

References

[1] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019.

[2] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global Vectors for Word Representation," in *Proc. EMNLP*, 2014.

[3] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," *arXiv preprint arXiv:1907.11692*, 2019.

[4] Z. Hartvigsen, E. Wallace, S. Singh, and M. Gardner, "ToxiGen: A Large-Scale Machine-Generated Dataset for Adversarial and Implicit Hate Speech Detection," in *Proc. ACL*, 2022.

[5] Kaggle Hate Speech Dataset. [Online]. Available: https://www.kaggle.com/datasets/waalbannyantudre/hate-speech-detection-curated-datase

[6] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in *Proc. EMNLP: System Demonstrations*, 2020.

[7] S. Montanari, "FastAPI: Modern Web Framework for Building APIs with Python 3.6+," [Online]. Available: https://fastapi.tiangolo.com